



UMĚLÁ INTELIGENCE

Přednáška 7
Strojové učení

Klasické metody strojového učení lze definovat jako soubor algoritmů a technik, které umožňují počítačovým systémům učit se z dat a vylepšovat své výkony bez explicitního programování pro konkrétní úlohy. Tyto metody se dělí do tří základních kategorií:

- **Učení s učitelem** (supervised learning): Modely se učí na množině dat (vstupů), pro kterou jsou známy správné odpovědi (výstupy). Cílem je naučit se předpovídat výstupy na základě vstupních dat. Základními úlohami jsou klasifikace a regrese.
- **Učení bez učitele** (unsupervised learning): Modely pracují s daty, u kterých nejsou známy správné odpovědi. Cílem je najít skryté struktury nebo vzory v datech. Základní úlohou je shlukování (clustering).
- **Posilované učení** (reinforcement learning): Modely se učí na základě odměn a trestů z prostředí, ve kterém operují, s cílem maximalizovat kumulativní odměnu.

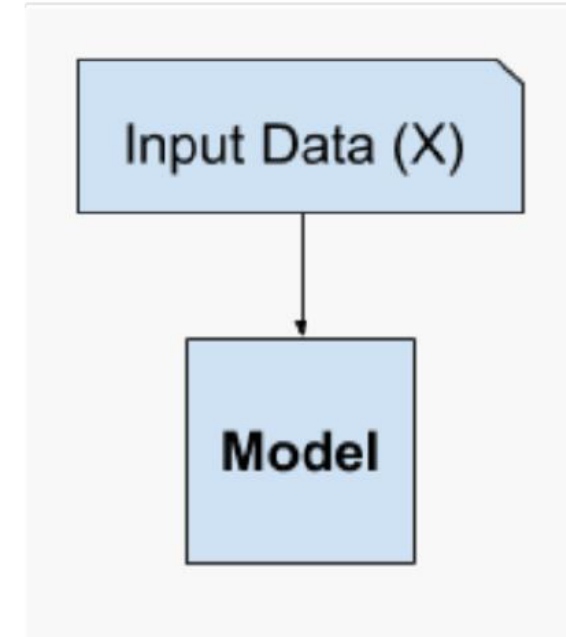
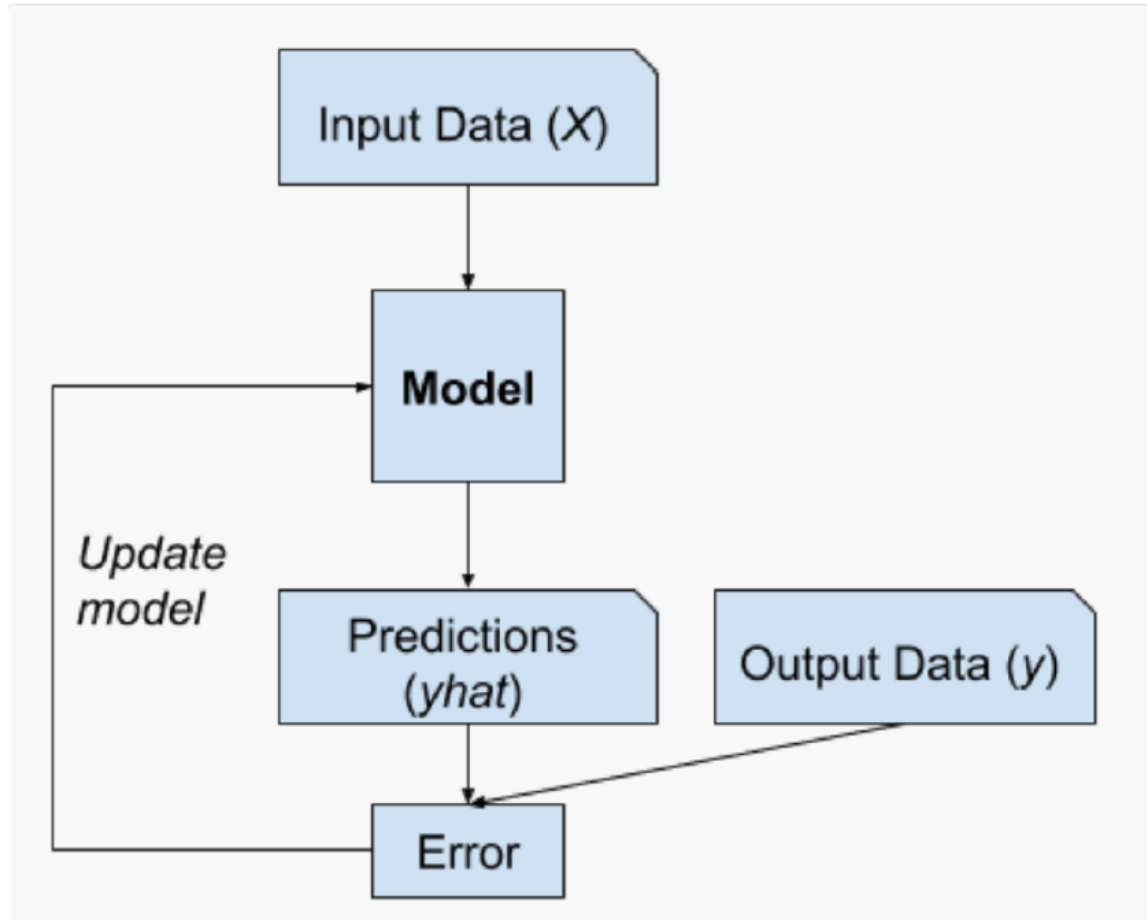
Úlohy strojového učení lze popsat pomocí **učícího se agenta**.

Agent pozoruje data, na jejich základě vytváří model a tento model používá k rozhodování při řešení problémů

OBECNÉ SCHÉMA STROJOVÉHO UČENÍ

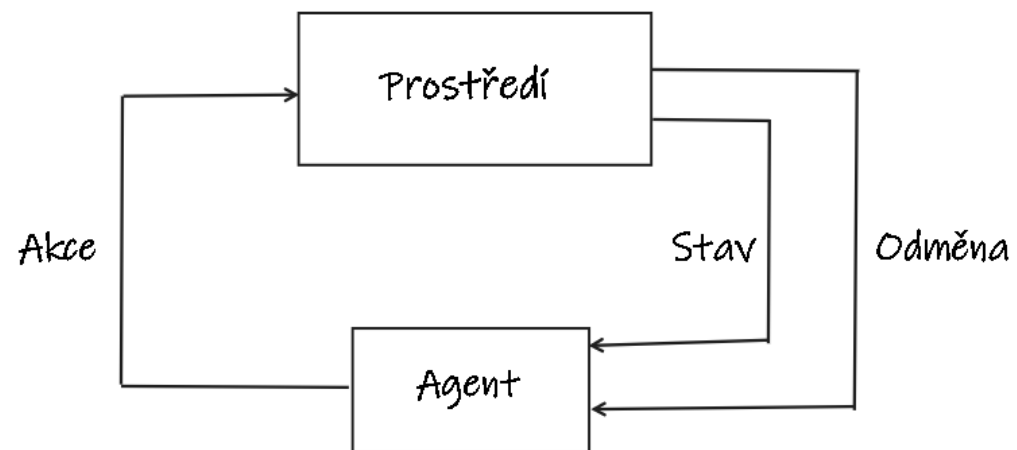
- identifikace problému – co řešíme – klasifikace obrázků, dokumentů, predikce cen nemovitostí, detekce spamu, doporučování produktů, konverze řeči na text, generování obrázků, textu, videa, ...
 - získání a úprava dat – kontrola „čistoty“, úplnosti, případně normalizace, škálování, výběr nebo sdružování vstupů (neboli příznaků, atributů, vlastností)
 - vytvoření modelu (neboli hypotézy, učící funkce, predikční funkce); tato matematická funkce popisuje pozorovaná data, a později je použita na předpovědi výstupů pro nová vstupní data
 - použití modelu na definovanou úlohu
-

Schéma učení s učitelem (vlevo) a bez učitele (vpravo)



zdroj: <https://machinelearningmastery.com/what-are-generative-adversarial-networks-gans/>

Posilované učení



UČENÍ S UČITELEM

- je zadána množina N vzorů, tj. dvojic vstup-výstup $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$
 - vstup – vektor dat
 - výstup:
 - jedna hodnota z množiny hodnot --> **klasifikace**
 - číslo --> **regrese**
 - tyto dvojice (vstup, výstup) jsou generovány neznámou funkcí **f**: $y = f(x)$
 - cílem je najít hypotézu **h**, která bude nejlépe aproximovat funkci **f**
 - hypotézy se vybírají z množiny hypotéz **H**
-

Příklady hypotéz (funkcí), které nejlépe odpovídají datům

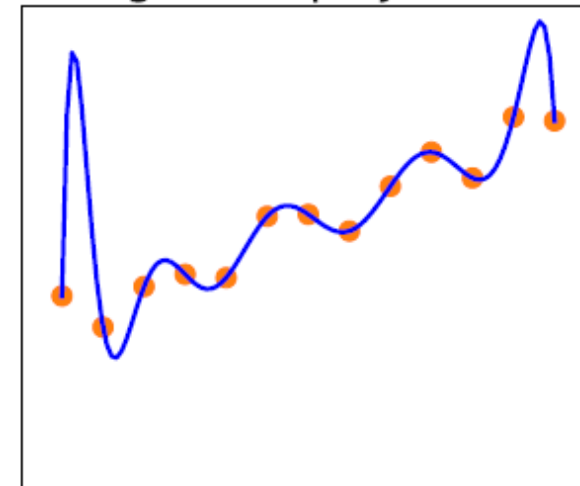
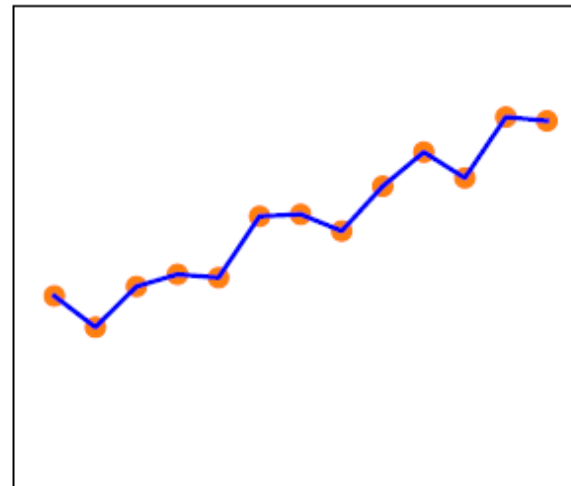
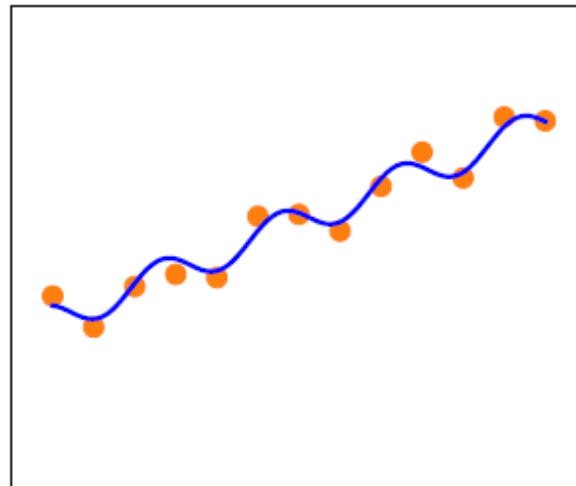
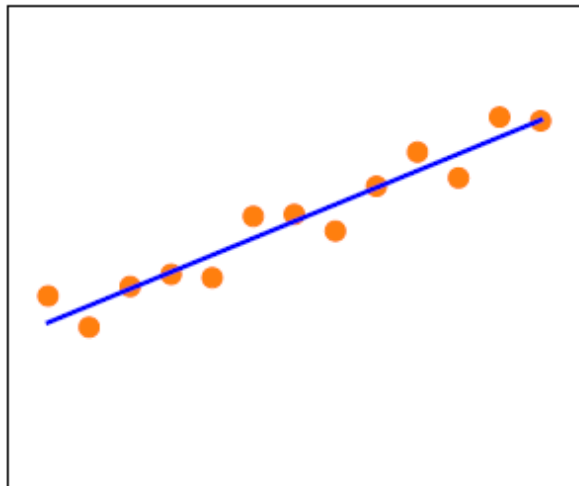
Linear

Sinusoidal

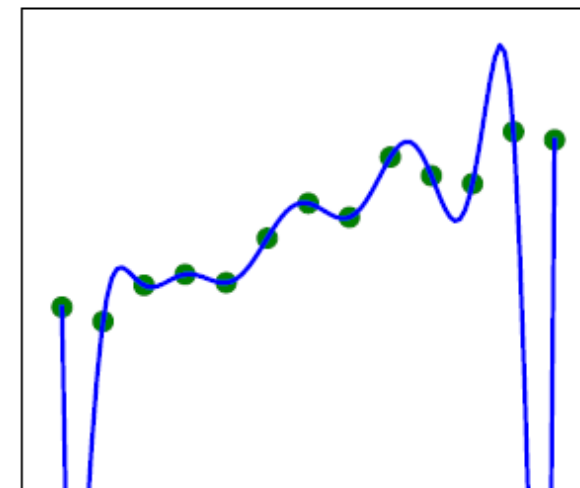
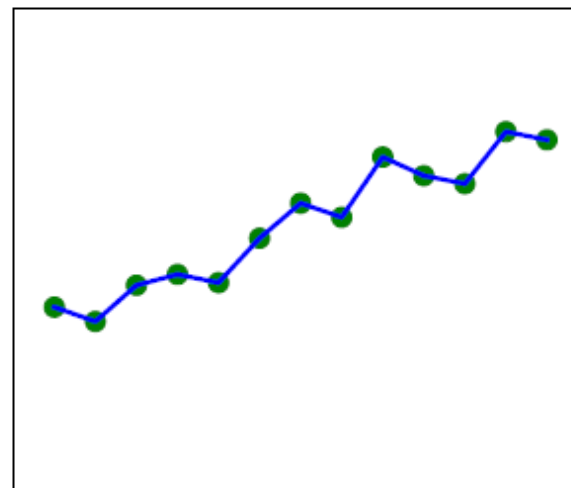
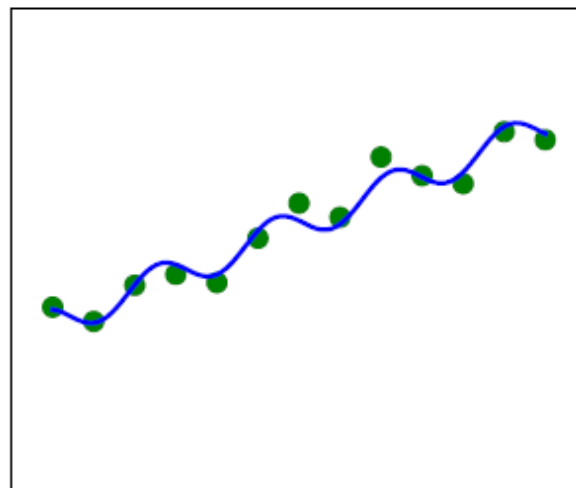
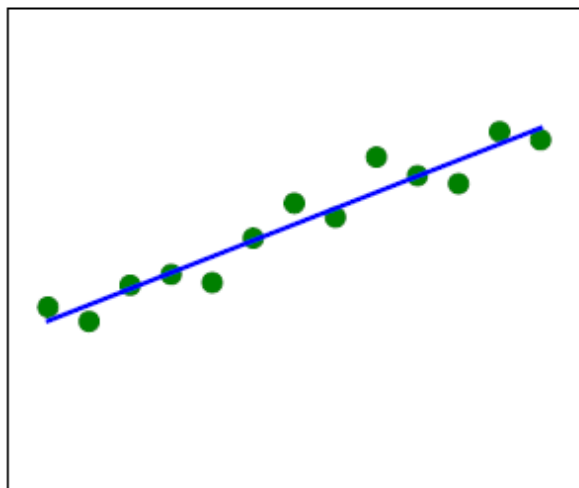
Piecewise linear

Degree-12 polynomial

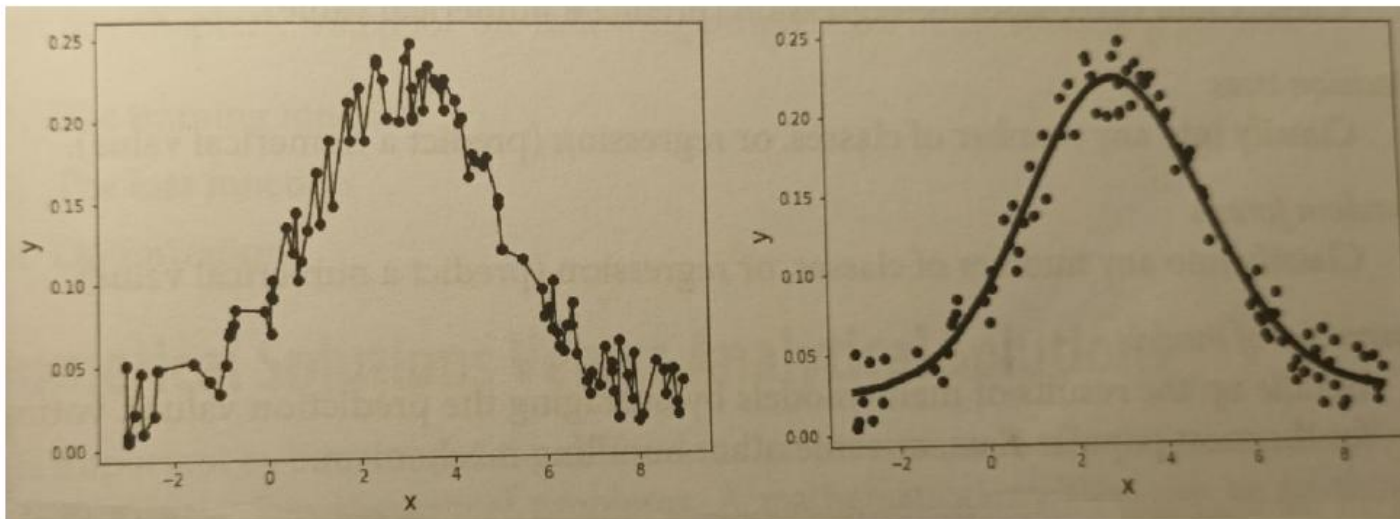
Data set 1



Data set 2



UČENÍ S UČITELEM



vlevo – příklad přeučení („overfitting“) - funkce přesně odpovídá trénovacím datům,
ale není dobrá pro predikci s novými daty
vpravo – vhodně zvolený model zohledňující šum v datech

UČENÍ S UČITELEM – ZÁKLADNÍ POJMY

- **trénovací množina** je soubor dat se známými výstupy, který agent používá, aby se naučil předpovídat další výstupy na základě zadaných vstupů; slouží pro výběr hypotézy
 - **validační množina** je sada dat, která se používá k doladění parametrů modelu a k prevenci přeučení
 - **testovací množina** je soubor dat se známými výstupy, který se používá k ověření přesnosti natrénovaného modelu, přičemž tato data nebyla použita pro trénování modelu
 - **konzistentní hypotéza** je hypotéza, která správně predikuje všechny příklady v trénovací množině, tj. pro všechna x_i v trénovací množině je výsledek $h(x_i) = y_i$
 - **generalizace** je schopnost modelu dobře předpovídat výstupy pro testovací množinu
 - **přeučení** (overfitting) nastává, když model příliš přesně odpovídá trénovací množině a ztrácí schopnost generalizace na nová data
-

UČENÍ S UČITELEM

Metody:

- lineární regrese – predikce (předpověď) čísla
- logistická regrese – klasifikace do 2 tříd (binární klasifikace)
- softmax regrese – klasifikace do více tříd
- SVM – support vector machines - předpověď čísla nebo binární klasifikace
- rozhodovací stromy - předpověď čísla nebo klasifikace do libovolného počtu tříd
- metoda nejbližších sousedů - klasifikace

Pro každou metodu je třeba:

- model – poskytuje výstup (odpověď) na základě vstupních dat
 - ztrátová funkce – popisuje, jak se liší předpovězený výstup od skutečného;
 - optimalizace – nalezení nejlepšího dostupného řešení – spočívá v minimalizaci hodnoty ztrátové funkce nastavováním parametrů modelu;
cílem je, aby model předpovídal s co nejmenší chybou
-

Výběr nejpravděpodobnější hypotézy pro daná data:
z množiny hypotéz H vybíráme takovou hypotézu h^* ,
která má pro zadaná data největší pravděpodobnost

$$h^* = \operatorname{argmax}_{h \in \mathcal{H}} P(h|data)$$

Bayesovo pravidlo - do výběru hypotézy vstupuje kromě dat i naše
apriorní znalost $P(h)$

$$h^* = \operatorname{argmax}_{h \in \mathcal{H}} P(data|h) P(h)$$

UČÍCÍ SE ROZHODOVACÍ STROMY

Učící se rozhodovací strom je algoritmus, který modeluje rozhodnutí a jejich možné důsledky pomocí stromové struktury. Každý uzel stromu představuje rozhodování podle jedné vybrané vlastnosti (atributu, příznaku) objektu. Z tohoto uzlu vede konečný počet hran, a proto je nutné vlastnosti měly diskrétní charakter. Spojité hodnoty je třeba diskretizovat (např. z reálných čísel do konečného počtu intervalů).

Učící algoritmus rozděluje trénovací data do jednotlivých tříd na základě vstupních příznaků. Začíná se u kořene stromu a pokračuje se rekurzivním dělením datové sady na menší podskupiny. Cílem je vytvořit model, který efektivně klasifikuje nová data na základě naučených pravidel.

Rozhodovací stromy se používají pro úlohy klasifikace i regrese.

Příklad: Přečte si člověk příspěvek na sociální síti?

Trénovací příklady:

Akce		Autor	Vlákno	Délka	Kde
e1	přeskakuje	známý	nové	dlouhé	doma
e2	čte	neznámý	nové	krátké	práce
e3	přeskakuje	neznámý	staré	dlouhé	práce
e4	přeskakuje	známý	staré	dlouhé	doma
e5	čte	známý	nové	krátké	doma
e6	přeskakuje	známý	staré	dlouhé	práce

Nové příklady:

e7	???	známý	nové	krátké	práce
e8	???	neznámý	nové	krátké	práce

UČÍCÍ SE ROZHODOVACÍ STROMY

Příklad – Rozhodování, zda počkat na místo v restauraci

- výstup – binární – odpověď ANO/NE
 - vstupy
 1. *Alternate*: whether there is a suitable alternative restaurant nearby.
 2. *Bar*: whether the restaurant has a comfortable bar area to wait in.
 3. *Fri/Sat*: true on Fridays and Saturdays.
 4. *Hungry*: whether we are hungry right now.
 5. *Patrons*: how many people are in the restaurant (values are *None*, *Some*, and *Full*).
 6. *Price*: the restaurant's price range (\$, \$\$, \$\$\$).
 7. *Raining*: whether it is raining outside.
 8. *Reservation*: whether we made a reservation.
 9. *Type*: the kind of restaurant (French, Italian, Thai, or burger).
 10. *WaitEstimate*: host's wait estimate: 0–10, 10–30, 30–60, or >60 minutes.
-

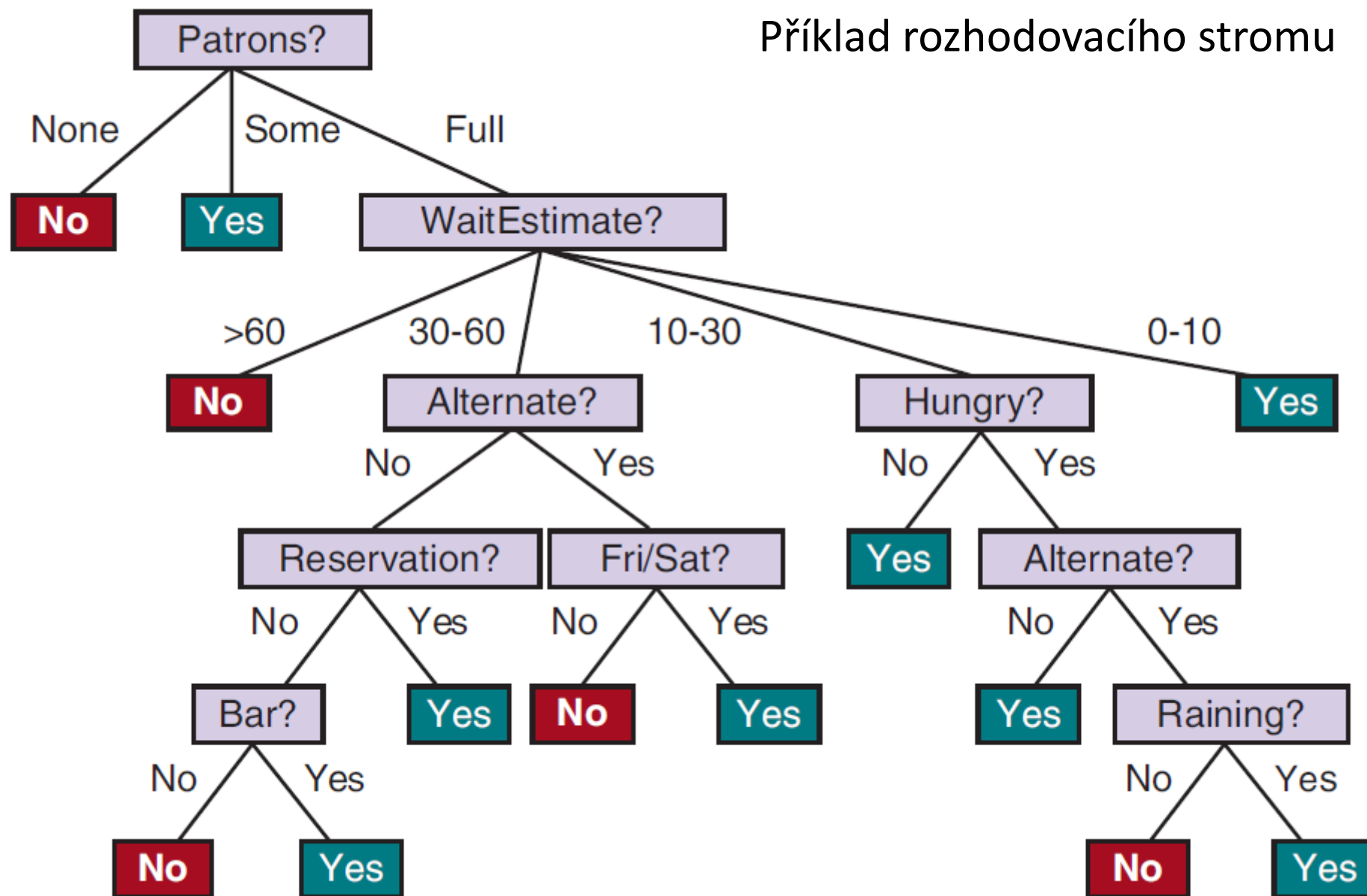
Trénovací množina [vstupy, výstup]

Example	Input Attributes										Output
	<i>Alt</i>	<i>Bar</i>	<i>Fri</i>	<i>Hun</i>	<i>Pat</i>	<i>Price</i>	<i>Rain</i>	<i>Res</i>	<i>Type</i>	<i>Est</i>	<i>WillWait</i>
x₁	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>French</i>	<i>0–10</i>	<i>y₁ = Yes</i>
x₂	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Thai</i>	<i>30–60</i>	<i>y₂ = No</i>
x₃	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Some</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Burger</i>	<i>0–10</i>	<i>y₃ = Yes</i>
x₄	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Thai</i>	<i>10–30</i>	<i>y₄ = Yes</i>
x₅	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>Full</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>French</i>	<i>>60</i>	<i>y₅ = No</i>
x₆	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$</i>	<i>Yes</i>	<i>Yes</i>	<i>Italian</i>	<i>0–10</i>	<i>y₆ = Yes</i>
x₇	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>None</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Burger</i>	<i>0–10</i>	<i>y₇ = No</i>
x₈	<i>No</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$</i>	<i>Yes</i>	<i>Yes</i>	<i>Thai</i>	<i>0–10</i>	<i>y₈ = Yes</i>
x₉	<i>No</i>	<i>Yes</i>	<i>Yes</i>	<i>No</i>	<i>Full</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Burger</i>	<i>>60</i>	<i>y₉ = No</i>
x₁₀	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>Italian</i>	<i>10–30</i>	<i>y₁₀ = No</i>
x₁₁	<i>No</i>	<i>No</i>	<i>No</i>	<i>No</i>	<i>None</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Thai</i>	<i>0–10</i>	<i>y₁₁ = No</i>
x₁₂	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Burger</i>	<i>30–60</i>	<i>y₁₂ = Yes</i>

UČÍCÍ SE ROZHODOVACÍ STROMY

- **rozhodnutí** = funkce, která vektoru hodnot vstupních atributů přiřadí výstupní hodnotu
 - **rozhodovací strom** - rozhodnutí je dosaženo prováděním řady testů počínaje kořenovým uzlem, dokud není dosaženo listového uzlu
 - každý vnitřní uzel stromu odpovídá testování jednoho vstupního atributu, větve z tohoto uzlu jsou označeny možnými hodnotami atributů, listy specifikují výstupní hodnotu
 - binární výstup --> Booleovská klasifikace
-

Příklad rozhodovacího stromu



Binární rozhodovací strom je ekvivalentní logickému výroku

$\text{výstup} \Leftrightarrow (\text{cesta}_1 \vee \text{cesta}_2 \vee \dots)$

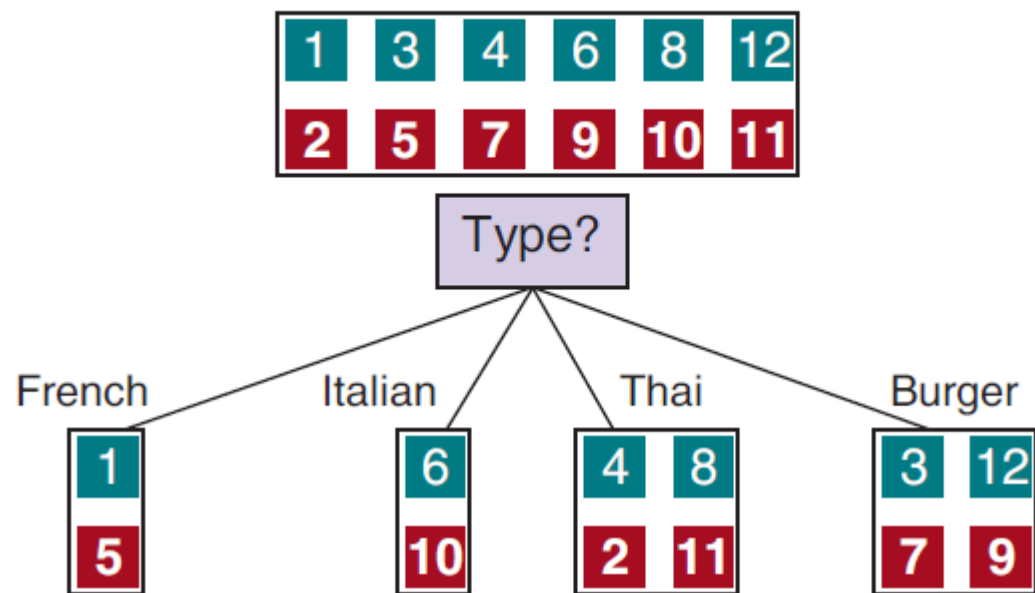
kde cesta_i je konjunkce hodnot atributů testu od kořene k pravdivému listu

$(A_1 = v_1 \wedge A_2 = v_2 \wedge \dots)$

Tedy celý výraz je v disjunktivní normální formě (DNF), a to znamená, že libovolná funkce (formule) výrokové logiky může být vyjádřena jako rozhodovací strom.

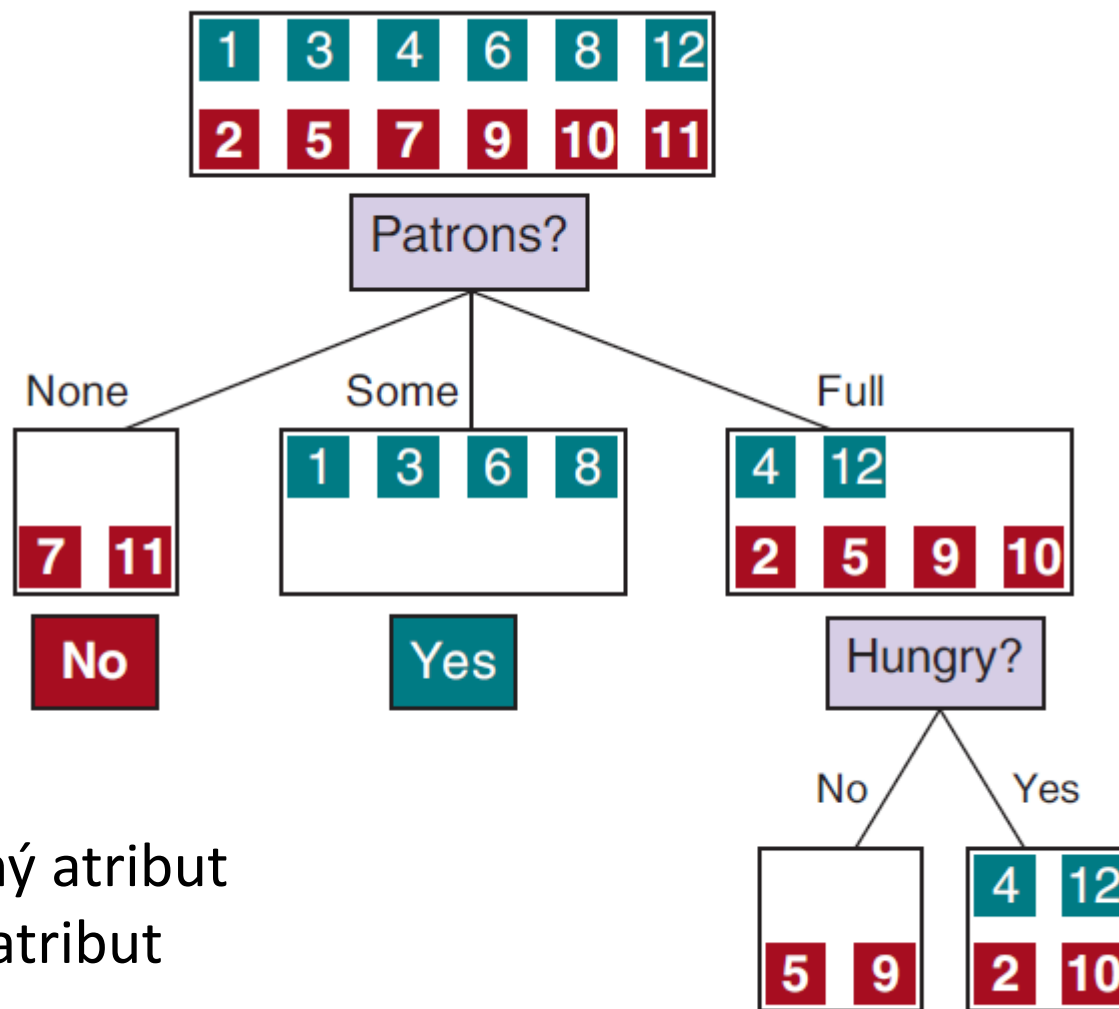
Příklad - sestavit rozhodovací strom, který odpovídá příkladu na předchozím snímku, a je co nejmenší.

Strategie - vždy nejprve otestovat nejdůležitější atribut a poté rekurzivně řešit menší podproblémy, které jsou definovány možnými výsledky testu. „Nejdůležitější atribut“ je takový, který má největší vliv na klasifikaci příkladu, a tedy výsledek klasifikace získáme pomocí menšího počtu testů než kdybychom začali testovat od „méně důležitého“ atributu.



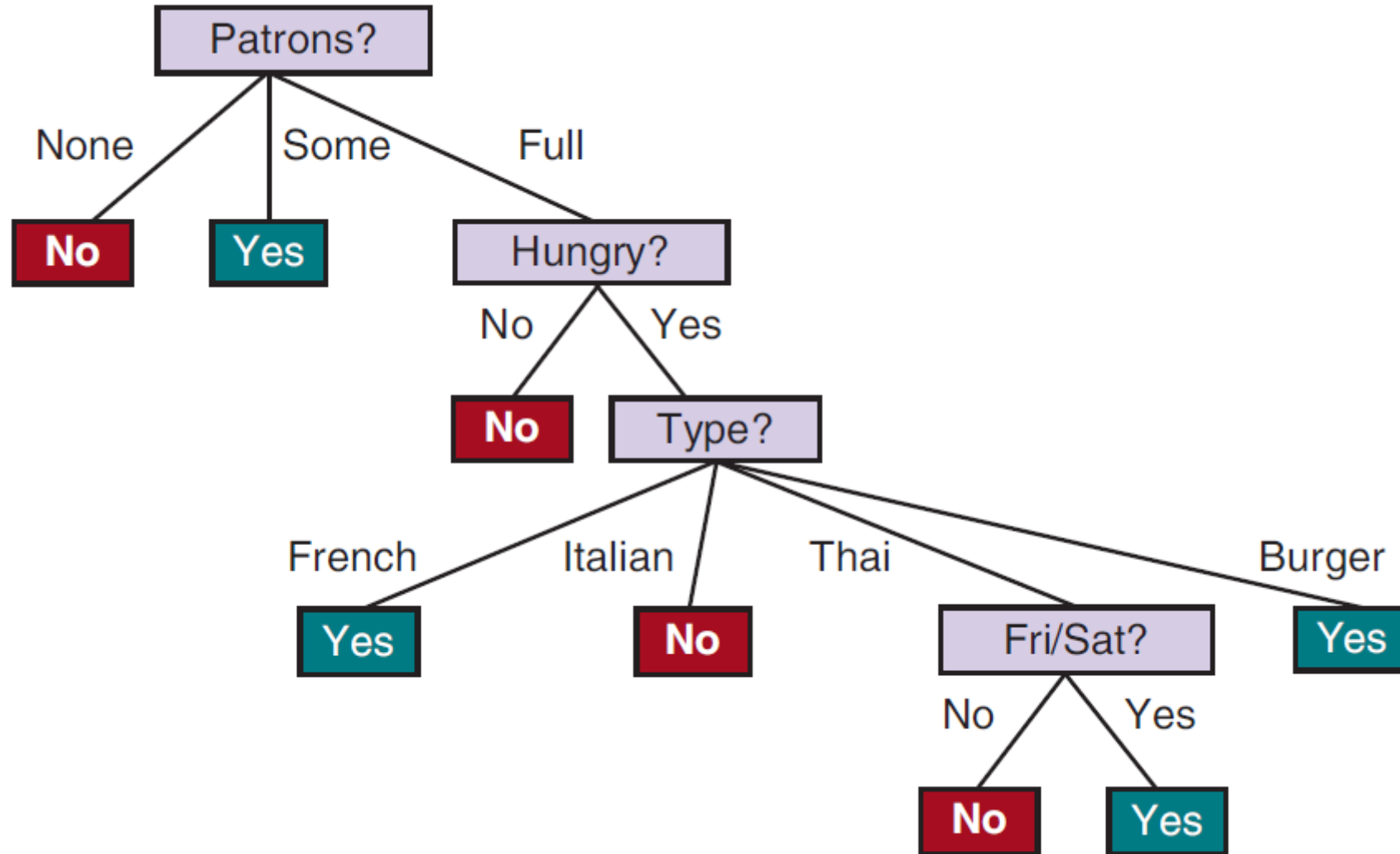
(a)

a) nevýznamný atribut
b) významný atribut

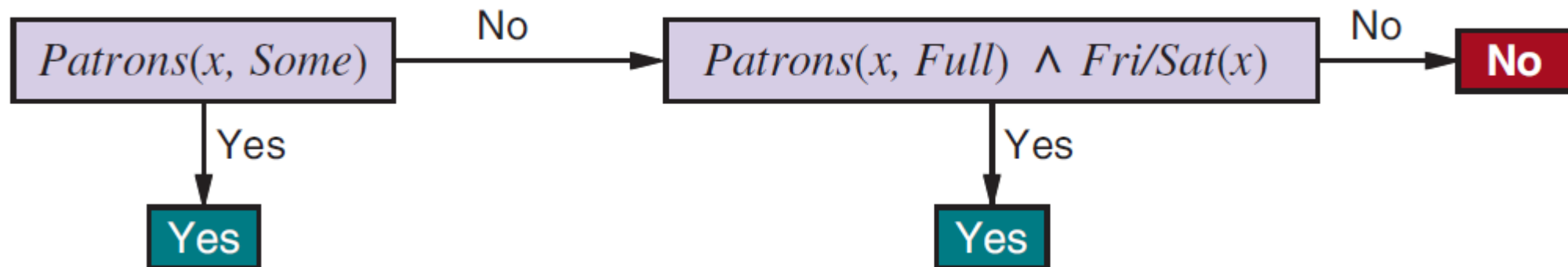


(b)

Výsledný optimální rozhodovací strom



Rozhodovací seznam - jednodušší forma rozhodovacího stromu



Rozhodovací seznam se skládá z řady testů, z nichž každý je konjunkcí literálů (atomických výroků). Pokud je test pro daný příklad úspěšný, rozhodovací seznam určí výsledné rozhodnutí. Pokud je test neúspěšný, pokračuje se dalším testem ze seznamu. Rozhodnutí se podobají rozhodovacím stromům, ale jejich struktura je jednodušší - větví se pouze jedním směrem. Na druhé straně ale jednotlivé testy jsou složitější.

Na obrázku je rozhodovací seznam, který reprezentuje následující hypotézu:

$WillWait \iff (Patrons = Some) \vee (Patrons = Full \wedge Fri/Sat)$

LINEÁRNÍ REGRESE A KLASIFIKACE

Regrese

- Jednorozměrná lineární regrese
- Sestupný gradient
- Vícerozměrná regrese

Klasifikace

- Lineární klasifikace s pevným prahem
 - Lineární klasifikace s logistickou regresí
-

Metody pro zjištění chybovosti modelu

- **k-násobná křížová validace** - statistická metoda používaná k ověření prediktivní schopnosti modelu, při které se datový soubor rozdělí na **k** částí, z nichž jedna slouží jako testovací sada a zbylé jako trénovací sady, a tento proces se opakuje **k**-krát; výsledkem je průměrná chybovost přes všechny iterace
 - **matice záměn** (confusion matrix) - vizualizace výkonu algoritmu klasifikace, která zobrazuje počty správných a nesprávných predikcí rozdělených podle tříd
 - **přesnost** (accuracy) - poměr správně klasifikovaných příkladů vůči celkovému počtu příkladů
 - **úplnost** (recall) - poměr správně identifikovaných pozitivních příkladů vůči celkovému počtu skutečných pozitivních příkladů
 - **preciznost** (precision) - poměr správně identifikovaných pozitivních příkladů vůči celkovému počtu pozitivních příkladů, které model identifikoval
 - pro regresní modely se často používají metriky jako **střední kvadratická chyba** (MSE) nebo **střední absolutní chyba** (MAE), které měří průměrnou chybu mezi predikovanými a skutečnými hodnotami
-

ZTRÁTOVÁ FUNKCE $L(y, \hat{y})$

Ztrátová funkce je metrika, která vyjadřuje, jak dobře model předpovídá skutečné výsledky. Měří rozdíl mezi skutečnými hodnotami y a hodnotami $\hat{y}$, které předpověděl model. Cílem trénování modelu je minimalizovat tuto funkci a tak zlepšit přesnost modelu.

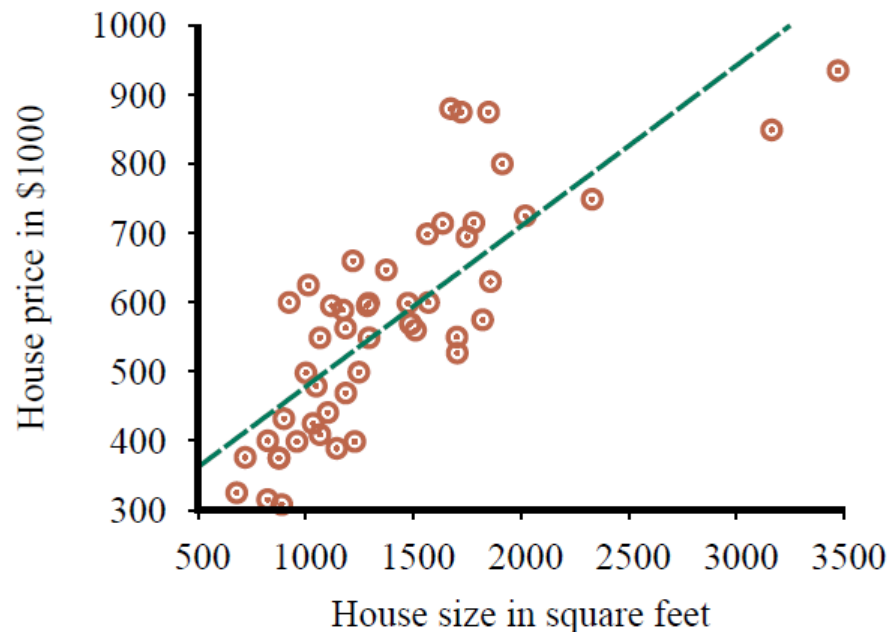
Pro regresní úlohy se často používá střední kvadratická chyba L_1 nebo střední absolutní chyba L_2 , pro klasifikační úlohy zase křížová entropie $L_{0/1}$.

$$L_1(y, \hat{y}) = |y - \hat{y}|$$

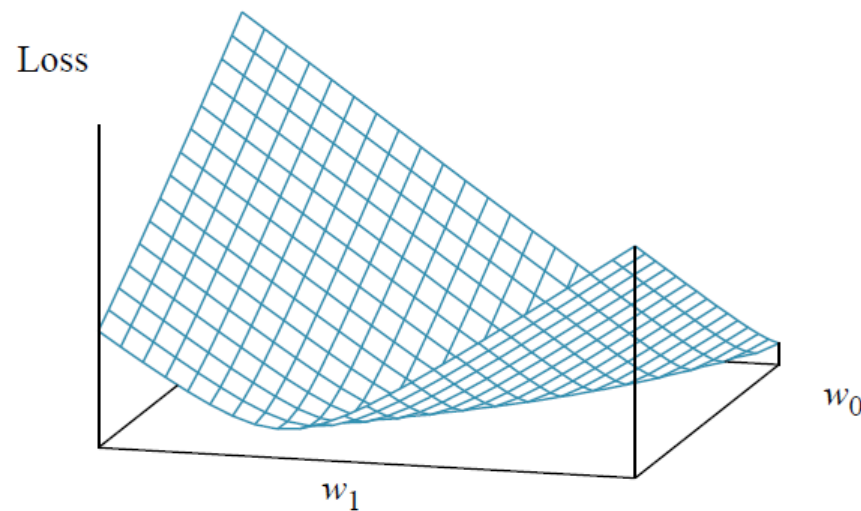
$$L_2(y, \hat{y}) = (y - \hat{y})^2$$

$$L_{0/1}(y, \hat{y}) = 0 \text{ if } y = \hat{y}, \text{ else } 1$$

Příklad jednorozměrné lineární regrese a kvadratické ztrátové funkce



(a)



(b)

(a) Data – cena domu v závislosti na podlahové ploše a lineární model, který minimalizuje střední kvadratickou odchylku: $y = 0,232x + 246$.

(b) Ztrátová funkce $\sum_j (y_j - w_1 x_j + w_0)^2$ (s jedním minimem)

LINEÁRNÍ REGRESE

- model $y = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$
- ztrátová funkce - střední kvadratická chyba $L = 1/N \sum_{i=1:N} (y_{\text{PREDICT}}^i - y_{\text{TRUE}}^i)^2$, N je počet dat
- optimalizace = nastavit váhy w_i tak, aby hodnota ztrátové funkce byla co nejmenší
- rozdělení označených dat:
 - trénovací množina – pro nastavování vah modelu
 - validační množina – pro ladění hyperparametrů (např. rychlost učení modelu), pro kontrolu přeučení modelu a/nebo pro porovnání s jinými metodami
 - testovací množina – data nepoužitá pro nastavení modelu, používá k otestování modelu před „vypuštěním do světa“

$$h_{\mathbf{w}}(\mathbf{x}_j) = w_0 + w_1x_{j,1} + \dots + w_nx_{j,n} = w_0 + \sum_i w_ix_{j,i} = \mathbf{w}^T \mathbf{x}_j$$

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}} \sum_j L_2(y_j, \mathbf{w} \cdot \mathbf{x}_j)$$

$$w_i \leftarrow w_i + \alpha \sum_j (y_j - h_{\mathbf{w}}(\mathbf{x}_j)) \times x_{j,i}$$

Vícerozměrná regrese

$$h_{\mathbf{w}}(\mathbf{x}_j) = w_0 + w_1x_{j,1} + \cdots + w_nx_{j,n} = w_0 + \sum_i w_ix_{j,i}.$$

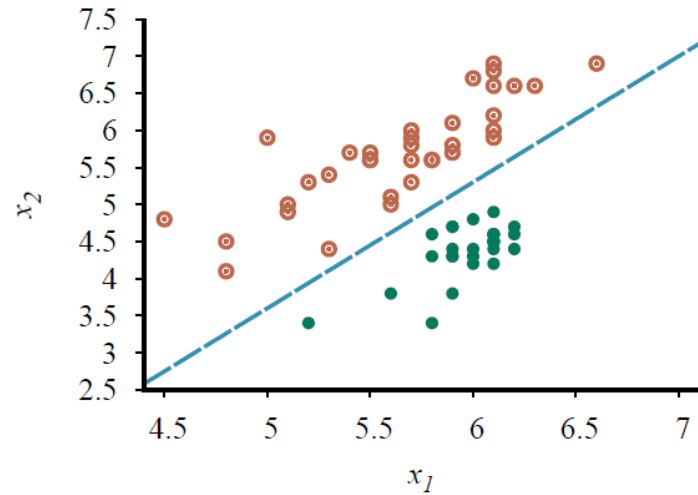
$$h_{\mathbf{w}}(\mathbf{x}_j) = \mathbf{w} \cdot \mathbf{x}_j = \mathbf{w}^\top \mathbf{x}_j = \sum_i w_ix_{j,i}.$$

$$\mathbf{w}^* = \operatorname{argmin}_{\mathbf{w}} \sum_j L_2(y_j, \mathbf{w} \cdot \mathbf{x}_j)$$

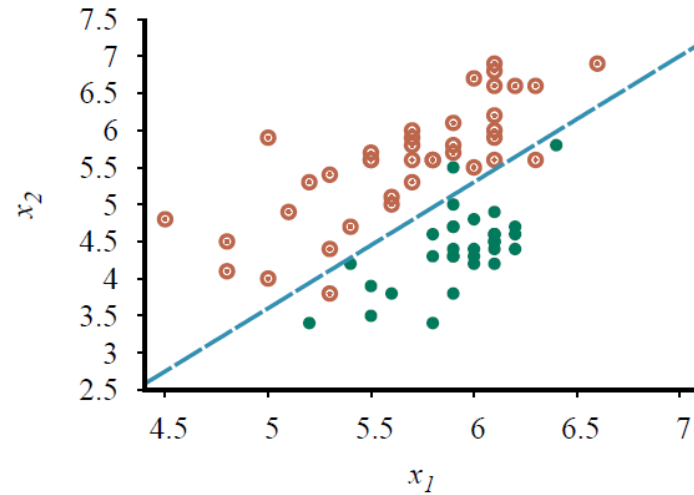
$$w_i \leftarrow w_i + \alpha \sum_j (y_j - h_{\mathbf{w}}(\mathbf{x}_j)) \times x_{j,i}$$

Lineární klasifikace s pevným prahem

- metoda strojového učení, která používá lineární rovnici pro předpověď výstupu
- pokud je výstupní hodnota rovnice vyšší než určitý práh, vstup je klasifikován do jedné třídy, v opačném případě je klasifikován do druhé třídy



(a)



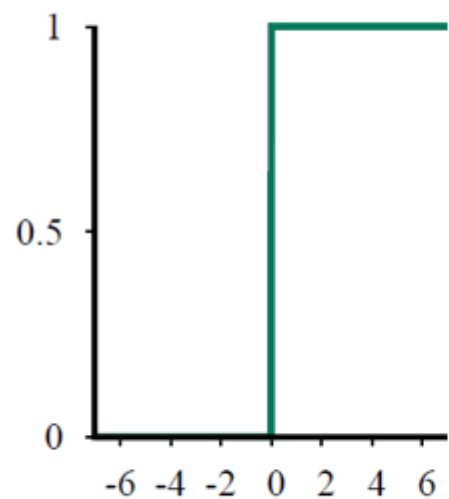
(b) nelze lineárně separovat

Lineární klasifikace s pevným prahem

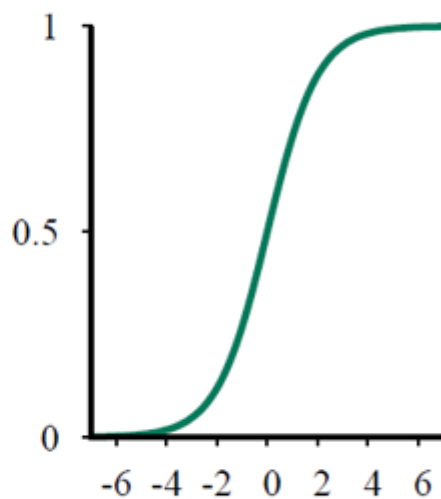
$h_{\mathbf{w}}(\mathbf{x}) = \text{Threshold}(\mathbf{w} \cdot \mathbf{x})$ where $\text{Threshold}(z) = 1$ if $z \geq 0$ and 0 otherwise.

- Učící pravidlo perceptronu

$$w_i \leftarrow w_i + \alpha (y - h_{\mathbf{w}}(\mathbf{x})) \times x_i$$



(a)



(b)

(a) Funkce s pevným prahem $\text{Threshold}(z)$ s výstupem 0/1.

(b) Logistická funkce, $\text{Logistic}(z) = \frac{1}{1 + e^{-z}}$
známá též jako sigmoidní funkce.

Lineární klasifikace s logistickou regresí

- metoda pro třídění dat do dvou kategorií
- výsledkem není odpověď ano/ne jako v případě klasifikace s pevným prahem, ale pravděpodobnost, s jakou objekt patří do dané třídy

$$h_{\mathbf{w}}(\mathbf{x}) = \text{Logistic}(\mathbf{w} \cdot \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w} \cdot \mathbf{x}}}$$

$$\text{Logistic}(z) = \frac{1}{1 + e^{-z}}$$

Nastavení vah – parametrů modelu

$$w_i \leftarrow w_i + \alpha (y - h_{\mathbf{w}}(\mathbf{x})) \times h_{\mathbf{w}}(\mathbf{x})(1 - h_{\mathbf{w}}(\mathbf{x})) \times x_i$$

NEPARAMETRICKÉ MODELY

Parametrické metody používají trénovací data k odhadu pevně dané množiny parametrů w , které určují výslednou hodnotu hypotézy $h_w(x)$, např. k odhadu vah w_0 a w_1 pro lineární regresi $h_w(x) = w_1 x + w_0$.

Po určení hodnot parametrů w pak můžeme trénovací data „zahodit“, protože jsou zobecněna pomocí těchto parametrů w .

Neparametrický model je takový, který nelze charakterizovat omezenou množinou parametrů.

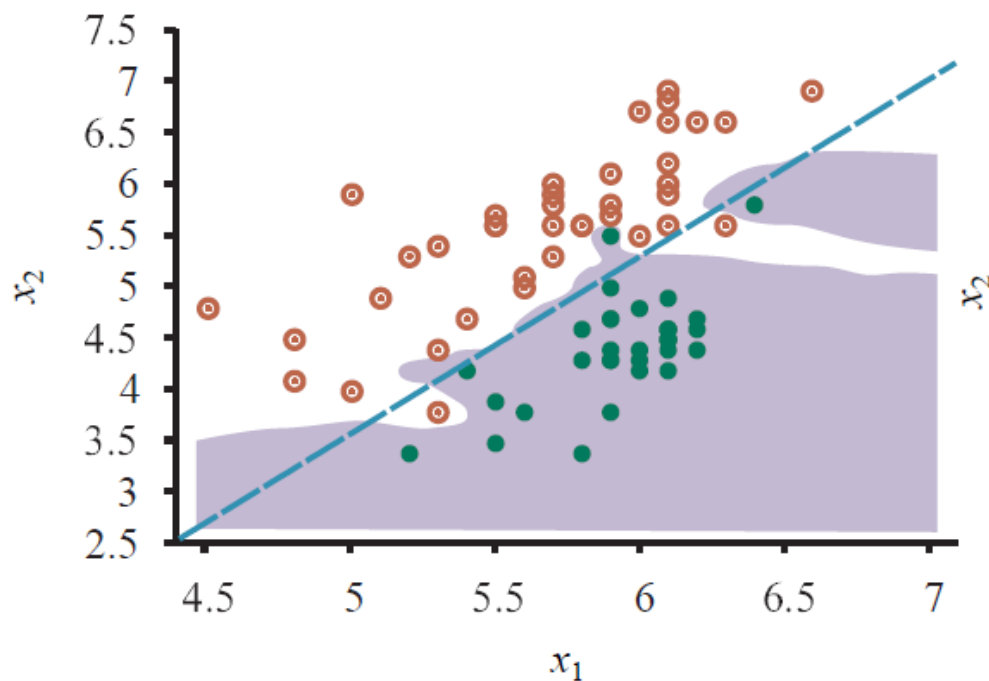
Trénovací data zůstávají v paměti a rozhodování probíhá s jejich využitím.

Metody:

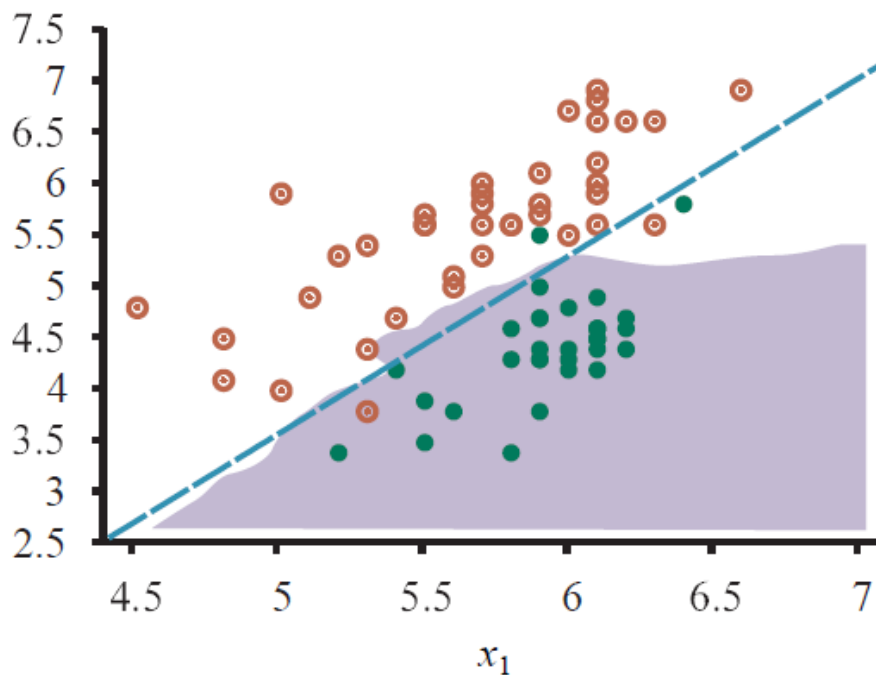
- Model nejbližších sousedů
 - Metoda SVM
-

MODEL NEJBLIŽŠÍCH SOUSEDŮ (k-NN)

- klasifikovaný objekt je zařazen do třídy, která je nejčastější mezi k nejbližšími sousedy v trénovací množině
- vzdálenost mezi objekty může být měřena různými způsoby, například Eukleidovskou vzdáleností
- volba počtu sousedů - příliš malé k může vést k přeučení
- metoda k-NN je užitečná v situacích, kde rozhodovací hranice mezi třídami je velmi nelineární



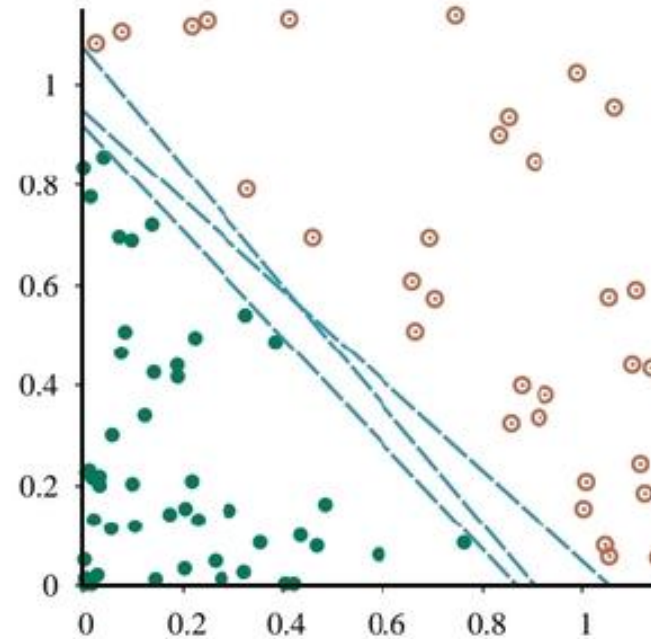
($k = 1$)



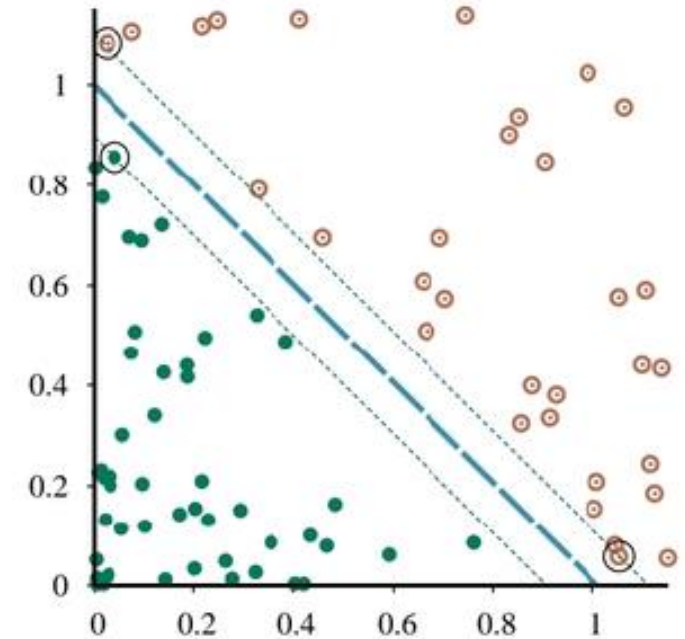
($k = 5$)

METODA SVM

- konstrukce maximálního separátoru – tj. rozhodovací hranice s největší možnou vzdáleností od vzorových příkladů → dobrá generalizace
- separační rovina je definována množinou vzorových příkladů, nikoli souborem hodnot parametrů
- porovnání: zatímco model nejbližších sousedů musí zachovat všechny vzorové příklady, model SVM zachovává pouze příklady, které jsou nejbližže separační rovině
- SVM kombinuje výhody neparametrických a parametrických modelů: má flexibilitu pro reprezentaci složitých funkcí, ale je odolná proti přeučení.



(a)

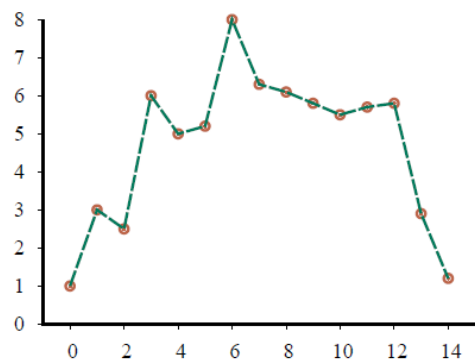


(b)

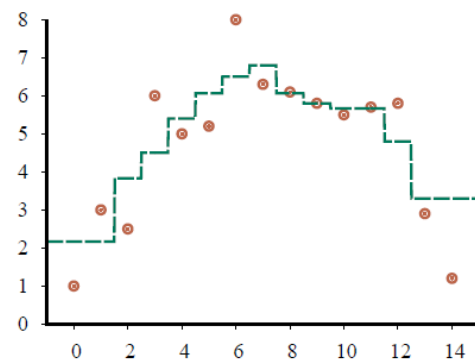
NEPARAMETRICKÉ REGRESNÍ MODELY

(a) spojování bodů, (b) 3 nejbližší sousedé - průměr

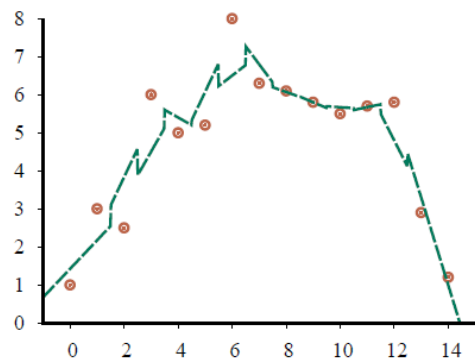
(c) lineární regrese se 3 nejbližšími sousedy, (d) lokálně vážená regrese s kvadratickým jádrem o šířce 10.



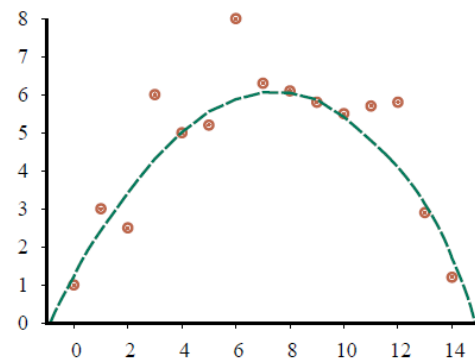
(a)



(b)



(c)



(d)

Zdroje:

Russell, S. J., Norvig, P. *Artificial Intelligence: A Modern Approach*. 4. vyd. Pearson, 2021

POOLE, D., L., MACKWORTH, A.K. *Artificial Intelligence: Foundations of Computational Agents*, 3rd Edition, 2023.

Dostupné online: <https://artint.info/>.
